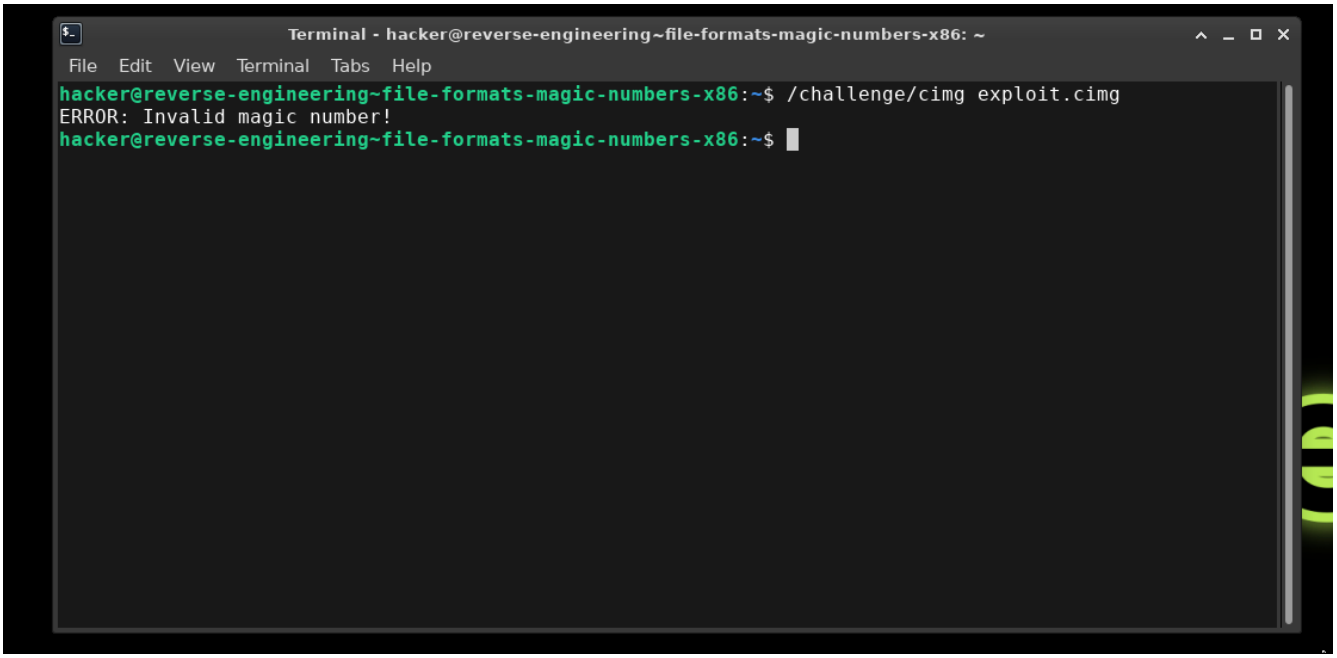
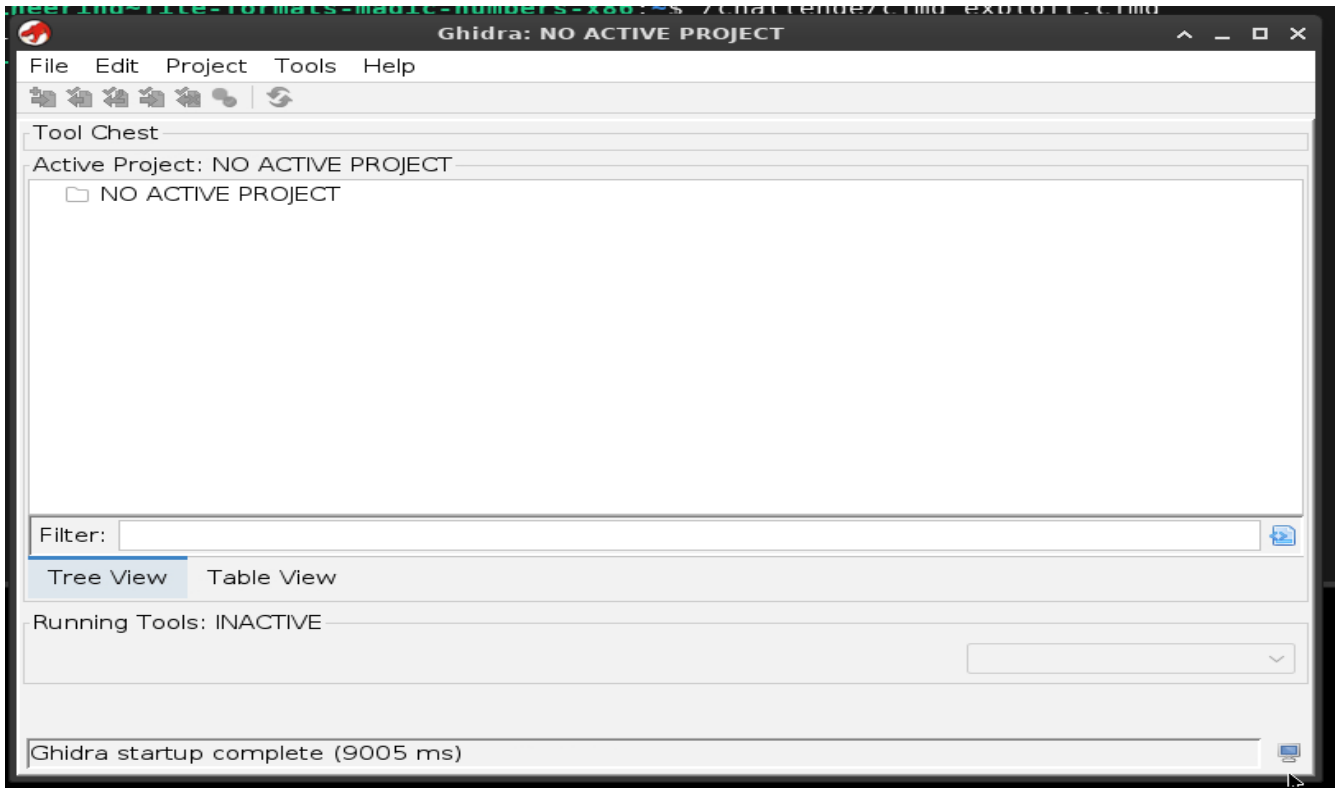
Reverse engineering (Ghidra)

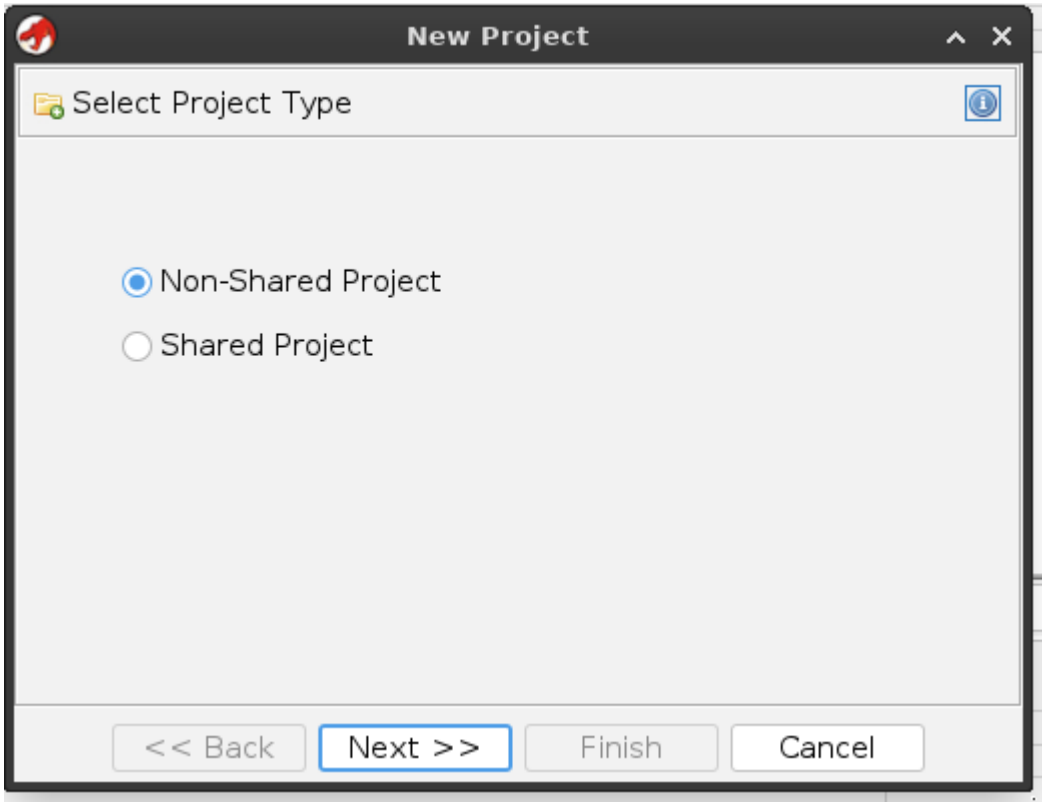
In this repo I will demonstrate how to use ghidra by prograssively solving ghidra pwn challenges.
First challenge I have to find a way to bypass this challenge by modifying file exploit.cimg.
I do it by using tool called ghidra that can disassemble:



When I open ghidra window like this will appear:



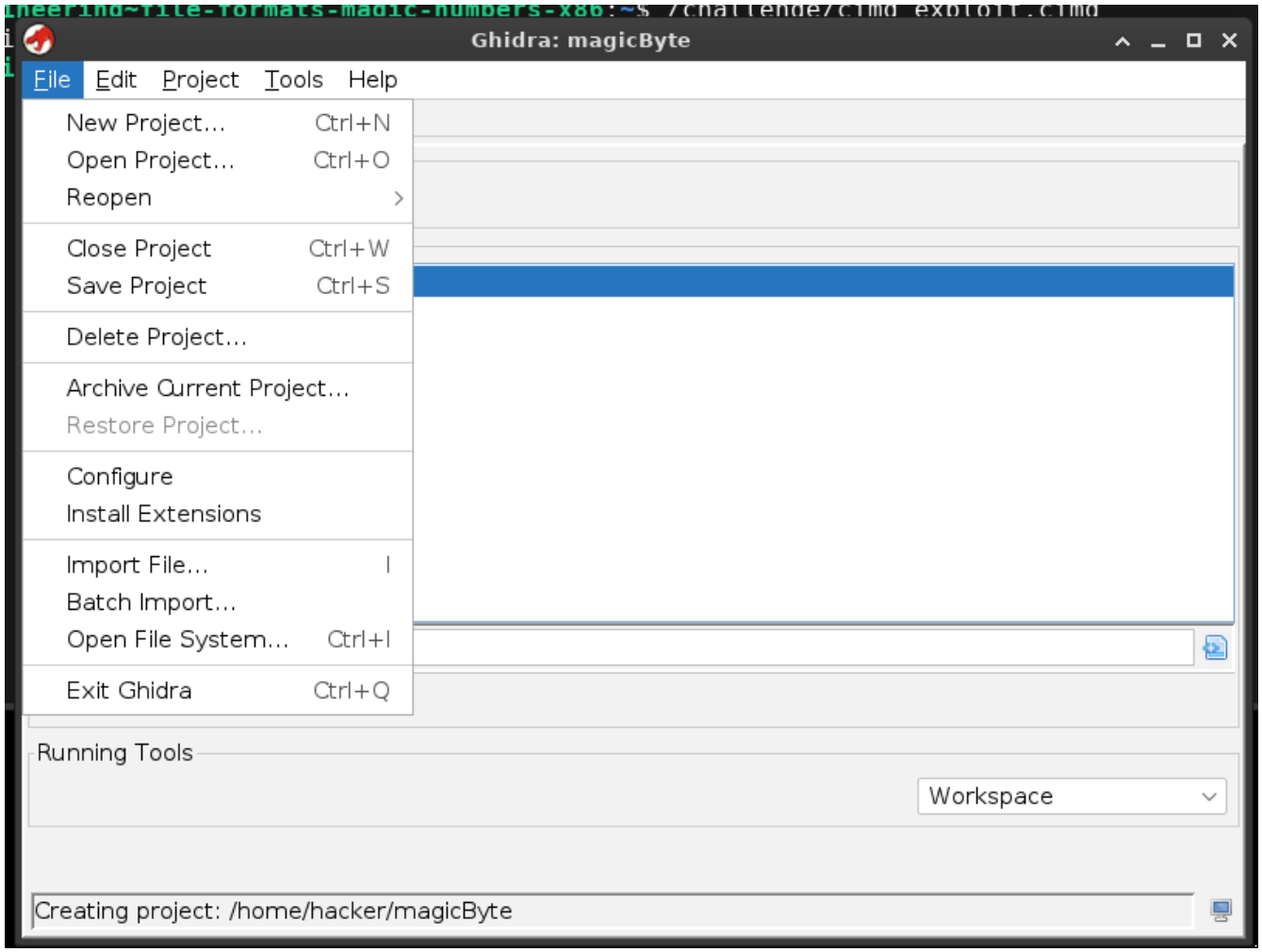
First I go to create new project and see window like this:



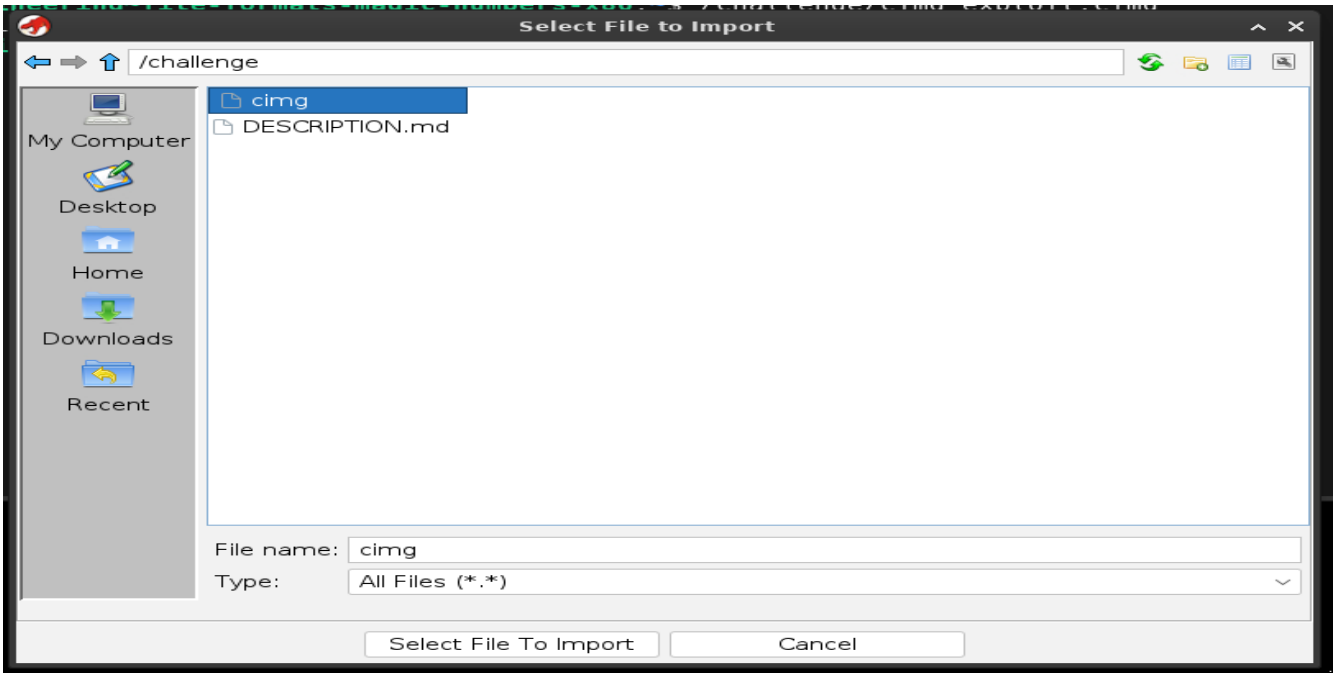
Then I will clikc on next and then enter name magicByte and click on finish.

Then I will see this window:

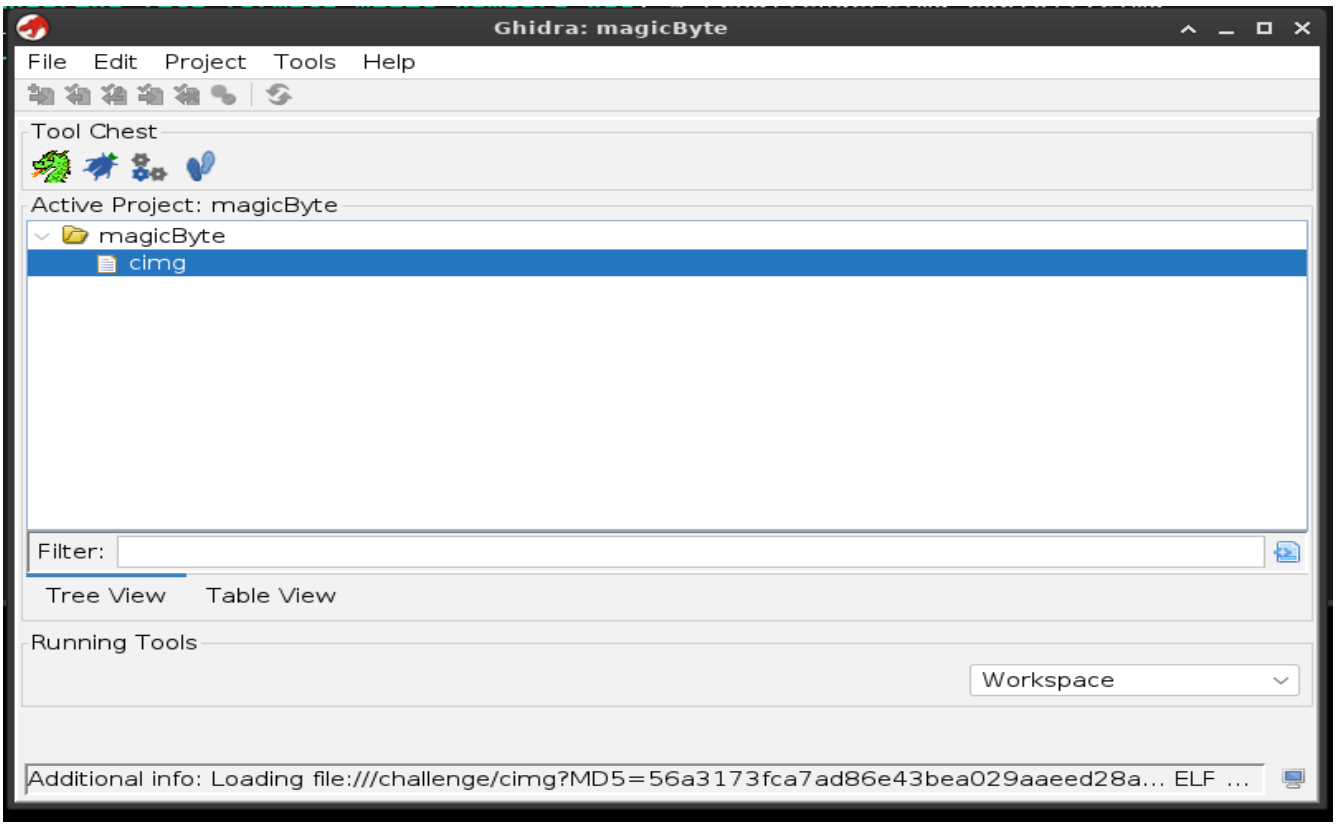
Then I will select file tab and select import file:



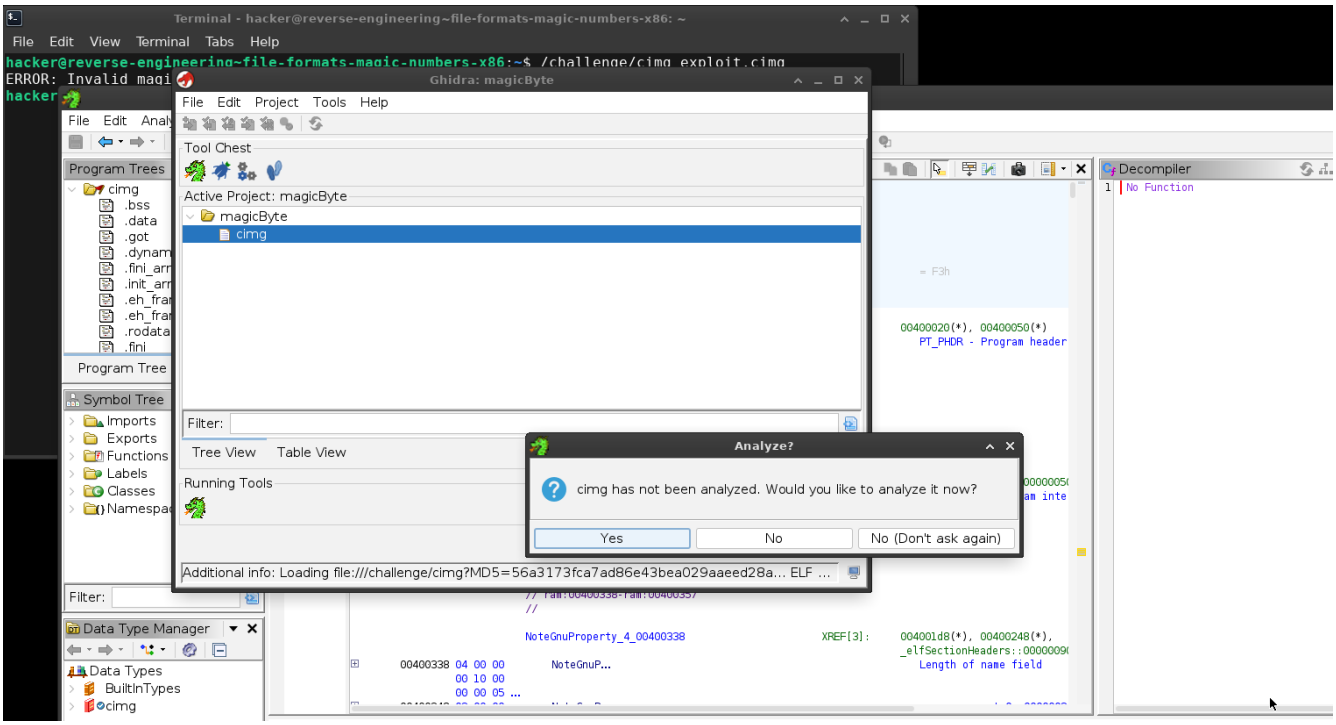
Then I will select challenge file that is used to verify file:



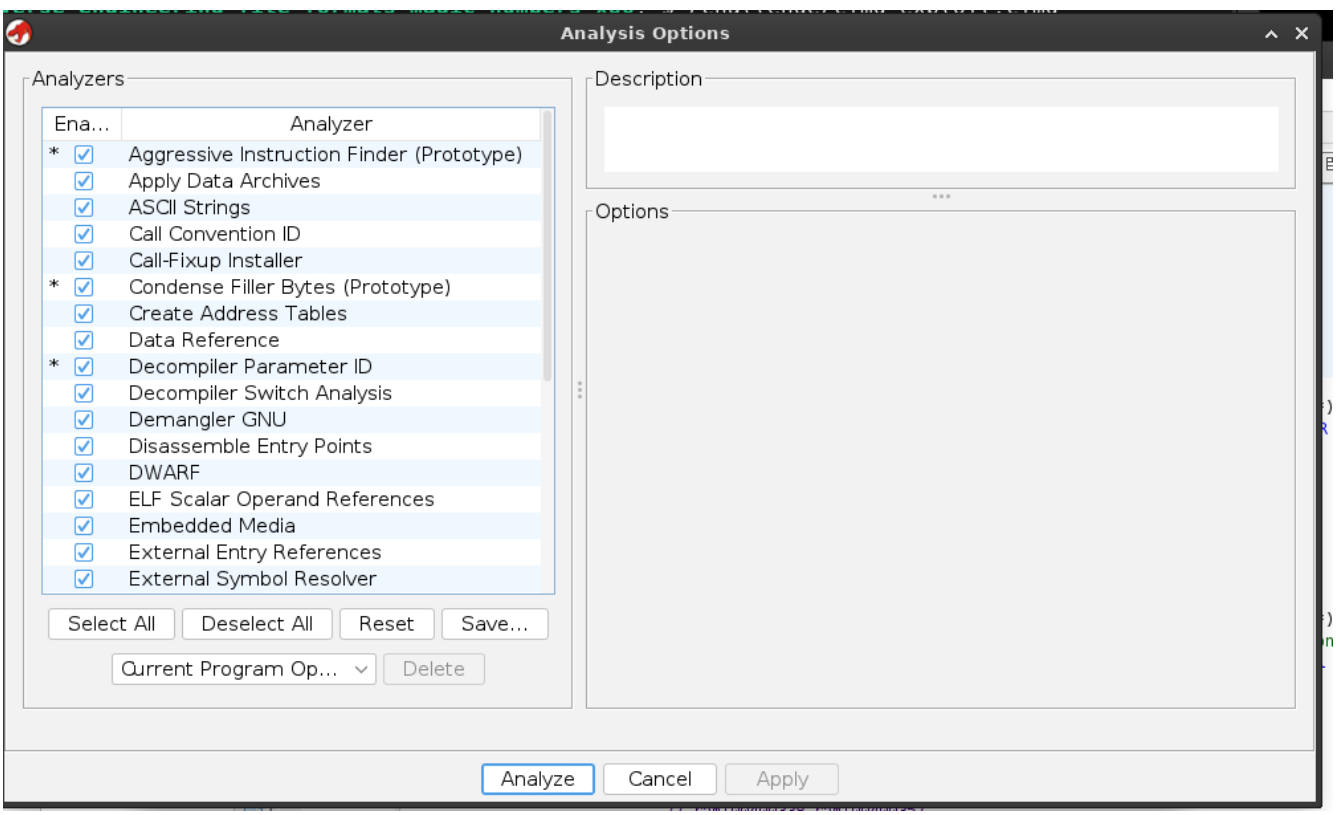
Then I will see window looking like that and I will click on file cimg:



Then in order analyze file I will click on yes:



Then I will click on select all options and apply and then analyze:



```
{
  long lVar1;
  int iVar2;
  size_t sVar3;
  long in_FS_OFFSET;
  undefined4 local_24;
  long local_20;

  local_20 = *(long *)(in_FS_OFFSET + 0x28);
  local_24 = 0;
  if (1 < param_1) {
    lVar1 = *(long *)(param_2 + 8);
    sVar3 = strlen(*(char **)(param_2 + 8));
    iVar2 = strcmp((char *)((sVar3 - 5) + lVar1),".cimg");
    if (iVar2 != 0) {
      printf("ERROR: Invalid file extension!");
                    /* WARNING: Subroutine does not return */
      exit(-1);
    }
    iVar2 = open(*(char **)(param_2 + 8),0);
    dup2(iVar2,0);
  }
  read_exact(0,&local_24,4,"ERROR: Failed to read header!",-1);
  if (((((char)local_24 == '<') && (local_24._1_1_ == 'M')) && (local_24._2_1_ == 'A')) &&
     (local_24._3_1_ == 'G')) {
    win();
    if (local_20 == *(long *)(in_FS_OFFSET + 0x28)) {
      return 0;
    }
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
  }
  puts("ERROR: Invalid magic number!");
                    /* WARNING: Subroutine does not return */
  exit(-1);
}
```

This is decompiled file and we can see that it checks if argument ends with .cimg extension and checks if first four bytes are equal to “<MAG”. I need to create file exploit.cimg which content will begin with “<MAG”. I do it with this command:

```
printf "<MAG" > exploit.cimg
```

Then when I enter this file as argument when running cimg:

```
hacker@reverse-engineering~file-formats-magic-numbers-x86:~$ /challenge/cimg exploit.cimg
pwn.college{Q26Xs2qxQJejmMbAuWyXRuNhNQK.0FMwMDMxwSO4gDM2EzW}
```